

算法分析与设计
Analysis and Design of Computer Algorithms
递归与分治策略

李纲
宁波大学
信息科学与工程学院

递归与分治策略



学习要点:

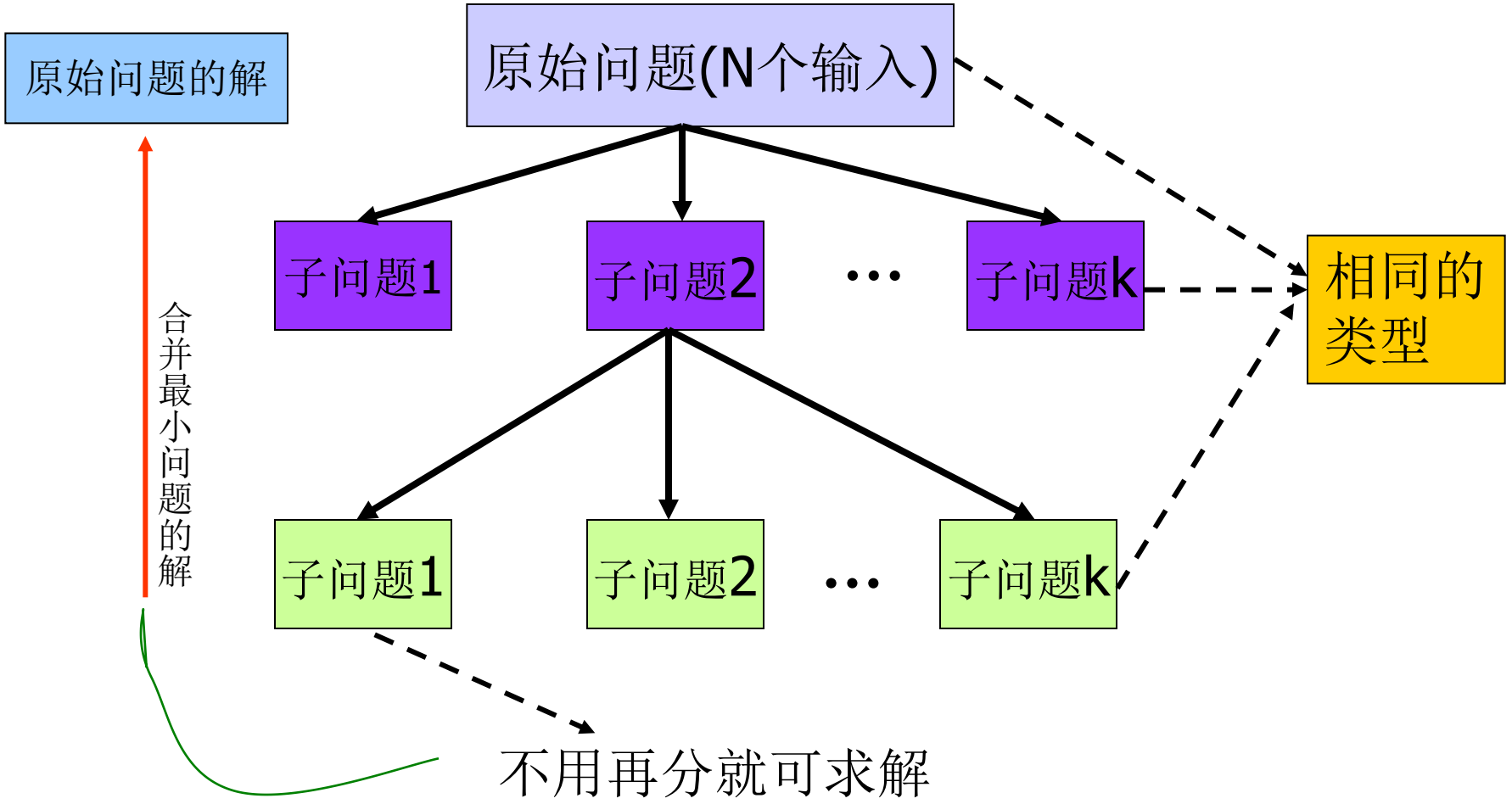
- 理解递归的概念。
- 掌握设计有效算法的分治策略。
- 通过下面的范例学习分治策略设计技巧。
- (1) 二分搜索技术;
- (2) 大整数乘法;
- (3) Strassen矩阵乘法;
- (4) 棋盘覆盖;
- (5) 合并排序和快速排序;
- (6) 线性时间选择;
- (7) 最接近点对问题;
- (8) 循环赛日程表。

递归与分治策略 的算法思想



- （1）将问题分解成k个子问题，对这k个子问题分别求解。如果子问题的规模仍然不够小，则再划分为k个子问题，如此递归的进行下去，直到问题规模足够小，很容易求出其解为止。
- （2）将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。
- 典型：归并排序

递归与分治策略 的算法思想





递归

- 直接或间接地调用自身的算法称为递归算法。用函数自身给出定义的函数称为递归函数。
- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的产生。
- 分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。
- （自顶向下+自底向上）

阶乘函数

阶乘函数可递归地定义为：

$$n!=\begin{cases}1 & n=0\\ n(n-1)! & n>0\end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

Fibonacci数列



无穷数列1，1，2，3，5，8，13，21，34，55，……，称为Fibonacci数列。它可以递归地定义为：

$$F(n)=\begin{cases}1 & n=0\\ 1 & n=1\\ F(n-1)+F(n-2) & n>1\end{cases}$$

边界条件

递归方程

第n个Fibonacci数可递归地计算如下：

```
int fibonacci(int n)
  {
     if (n <= 1) return 1;
     return fibonacci(n-1)+fibonacci(n-2);
  } //有的同学只有全部迭代出来才觉得自己理解了
```

尾递归 Tail recursion

尾递归不产生中间计算，将计算传递到下一层去

小明：老妈，我要花钱（老妈管钱）
老妈：问你爸同意不？（小明去问老爸）
小明：老爸说可以

小明：老爸，这个题不会做
老爸：找你妈

```
def recsum(x):
  if x == 1:
    return x
  else:
    return x + recsum(x - 1)
```

```
def tailrecsum(x, running_total=0):
  if x == 0:
    return running_total
  else:
    return tailrecsum(x - 1, running_total + x)
```

```
recsum(5)
5 + recsum(4)
5 + (4 + recsum(3))
5 + (4 + (3 + recsum(2)))
5 + (4 + (3 + (2 + recsum(1))))
5 + (4 + (3 + (2 + 1)))
5 + (4 + (3 + 3))
5 + (4 + 6)
5 + 10
15
```

```
tailrecsum(5, 0)
tailrecsum(4, 5)
tailrecsum(3, 9)
tailrecsum(2, 12)
tailrecsum(1, 14)
tailrecsum(0, 15)
15
```

《-------需要堆栈空间



Ackerman函数



当一个函数及它的一个变量是由函数自身定义时，称这个函数是**双递归函数**。

Ackerman函数A(n，m)定义如下：

$$
\begin{cases}
A(1,0) = 2 & \\
A(0,m) = 1 & m \geq 0 \\
A(n,0) = n+2 & n \geq 2 \\
A(n,m) = A(A(n-1,m),m-1) & n,m \geq 1
\end{cases}
$$

Ackerman函数无法找到非递归的定义。

Ackerman函数

$$\begin{cases} A(1,0)=2 & \\ A(0,m)=1 & m\geq 0 \\ A(n,0)=n+2 & n\geq 2 \\ A(n,m)=A(A(n-1,m),m-1) & n,m\geq 1 \end{cases}$$

- A(n，m)的自变量m的每一个值都定义了一个单变量函数：
- M=0时，A(n,0)=n+2
- M=1时，A(n,1)=A(A(n-1,1),0)=A(n-1,1)+2，和A(1,1)=2
- 故A(n,1)=2*n

A(1,1)=A(A(0,1),0)=A(1,0)=2

- M=2时，A(n,2)=A(A(n-1,2),1)=2A(n-1,2)，和A(1,2)=A(A(0,2),1)=A(1,1)=2，故A(n,2)= 2^n 。
- M=3时，类似的可以推出 $\underbrace{2^{2^{2^{\cdot^{\cdot^{\cdot 2}}}}}}_{n}$
- M=4时，A(n,4)的增长速度非常快，以至于没有适当的数学式子 来表示这一函数。



排列问题

设计一个递归算法生成n个元素$\{r_1,r_2,\ldots,r_n\}$的全排列。

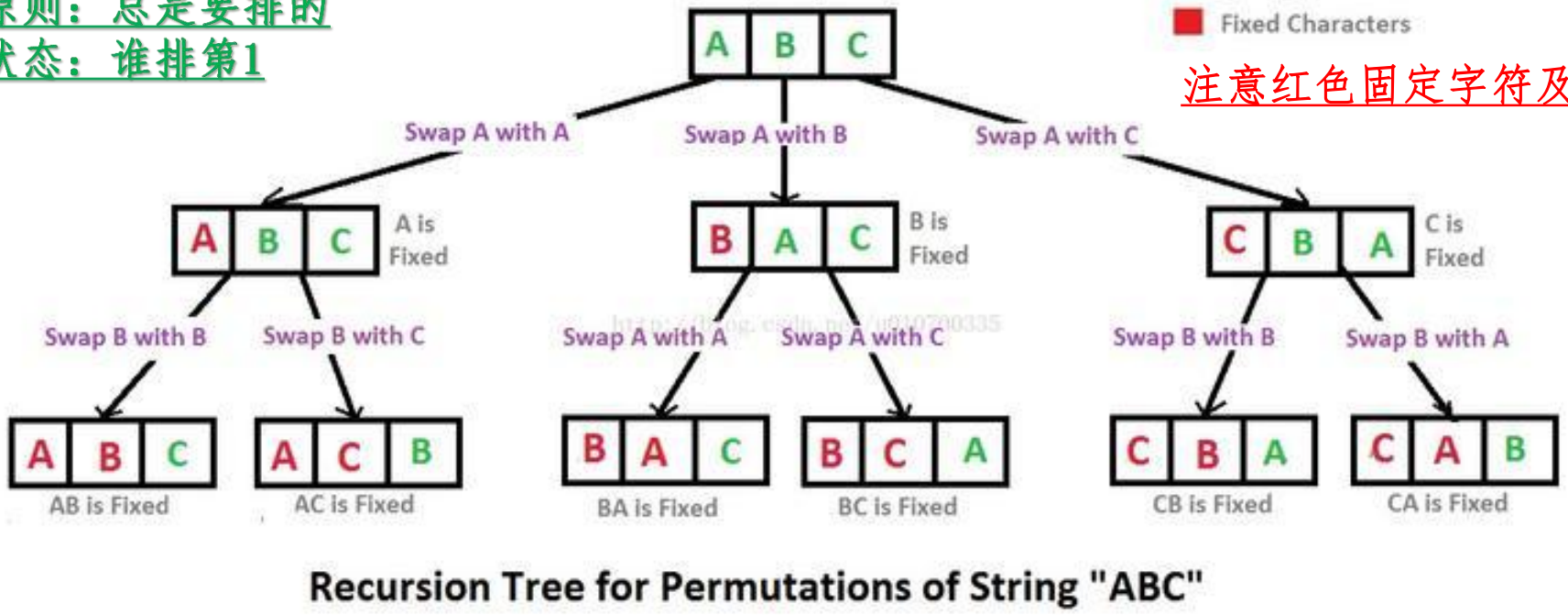
设$R=\{r_1,r_2,\ldots,r_n\}$是要进行排列的n个元素，$R_i=R-\{r_i\}$。
集合X中元素的全排列记为perm(X)。

(r_i)perm(X)表示在全排列perm(X)的每一个排列前加上前缀得到的排列。R的全排列可归纳定义如下：

当n=1时，perm(R)=(r)，其中r是集合R中唯一的元素；
当n>1时，perm(R)由(r_1)perm(R_1)，(r_2)perm(R_2)，…，(r_n)perm(R_n)构成。
注意教材中SWAP的复原问题，阅读P11

排列问题

原则：总是要排的
状态：谁排第1

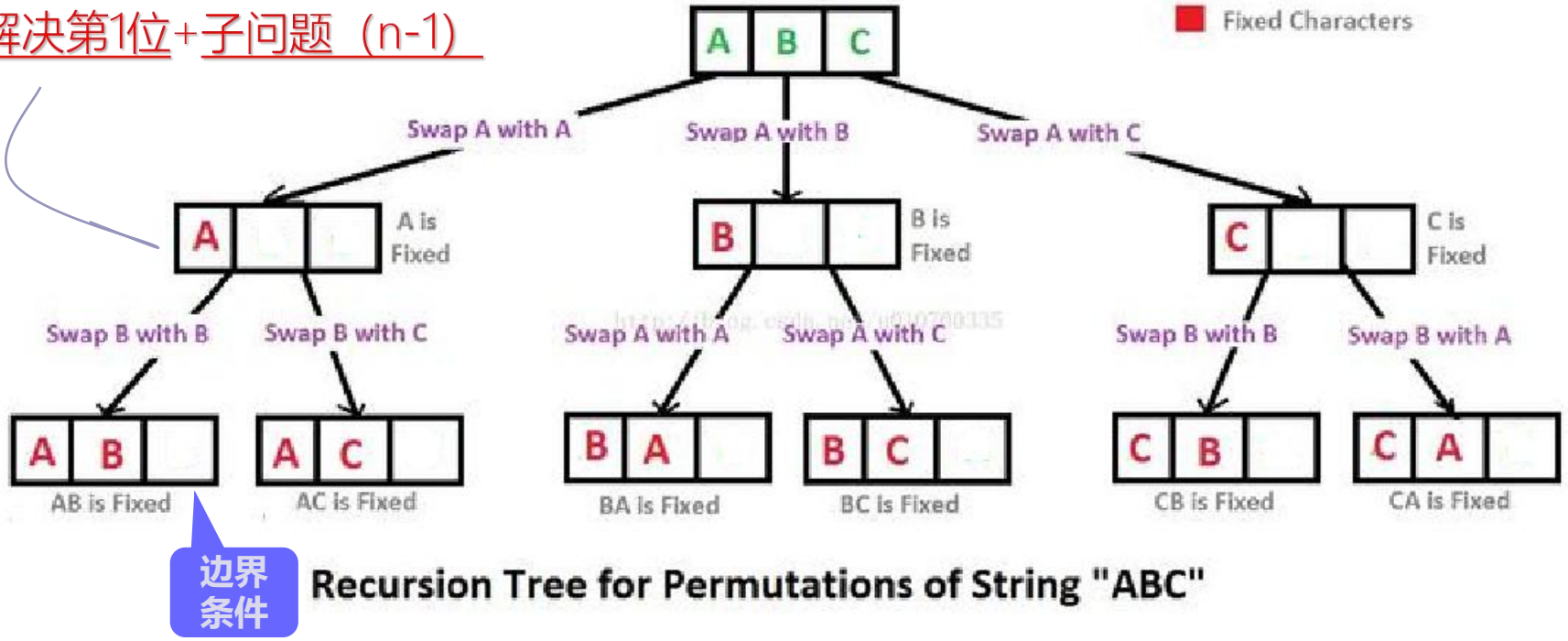


Recursion Tree for Permutations of String "ABC"

当n=1时，perm(R)=(r)，其中r是集合R中唯一的元素；
当n>1时，perm(R)由$(r_1)perm(R_1)$，$(r_2)perm(R_2)$，…，$(r_n)perm(R_n)$构成。

排列问题

解决第1位+子问题（n-1）

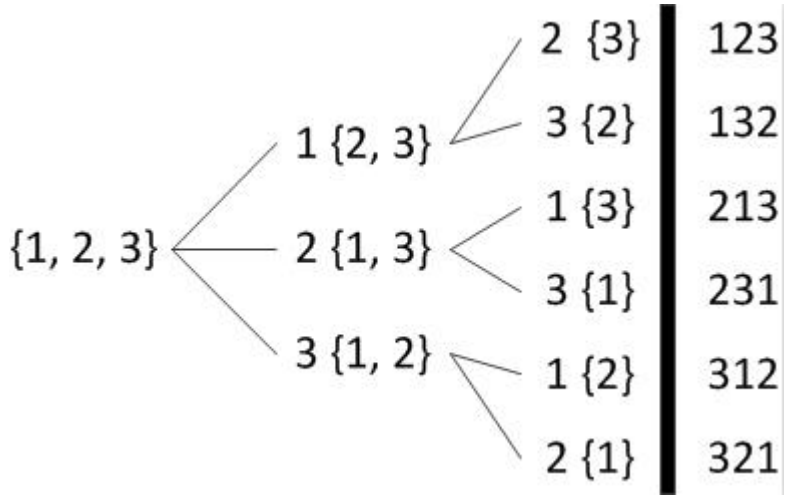


当n=1时，perm(R)=(r)，其中r是集合R中唯一的元素；
当n>1时，perm(R)由$(r_1)perm(R_1)$，$(r_2)perm(R_2)$，…，$(r_n)perm(R_n)$构成。

排列问题

```
int permutation(int *arr, int pos, int n)
{
    int i;
    if (pos == n) {
        print_array(arr, n);   //输出一组排列
        return 0;
    }

    for(i = pos; i < n; i++)
    {
        swap(arr, i, pos);                  //解决第1位：某个数字交换到 pos 位置/当前第1位置
        permutation(arr, pos + 1, n); //子问题：对出pos 位置后面的数，进行一次全排列
        swap(arr, i, pos);                  //复原原来的排列，准备交换下一个数到pos 位置。
    }
    return 0;
}
```



不恢复，递归返回会不会乱了？

也是一个深搜

整体结构就是：if 到底了；else 继续往下递归



整数划分问题

将正整数n表示成一系列正整数之和：$n=n_1+n_2+\ldots+n_k$，
其中$n_1\geq n_2\geq\ldots\geq n_k\geq 1$，$k\geq 1$。
求正整数n的不同划分个数。

例如正整数6有如下11种不同的划分：

6;
5+1;
4+2，4+1+1;
3+3，3+2+1，3+1+1+1;
2+2+2，2+2+1+1，2+1+1+1+1;
1+1+1+1+1+1。~~1+5，1+1+4，1+2+3~~

先排第一个数，+其他/绿色

如何不重复：大数在前



整数划分问题

在本例中，如果设p(n)为正整数n的划分数，则难以找到递归关系，因此考虑增加一个自变量：

将n最大加数不大于m的划分个数记作q(n,m)。

可以建立q(n,m)的如下递归关系。

(1) $q(n,1)=1, n\geq 1$;
当最大加数n_i不大于1时，任何正整数n只有一种划分形式，
即 $n=1+1+1+\cdots+1$

(2) $q(n,m)=q(n,n), 当m\geq n$;
最大加数n_i实际上不能大于n。
因此，$q(1,m)=1$。

整数划分问题

6:
...
2+2+2, 2+2+1+1, 2+1+1+1+1;
1+1+1+1+1+1。~~1+5, 1+1+4, 1+2+3~~

在本例中，如果设p(n)为正整数n的划分数，则难以找到递归关系，因此考虑增加一个自变量：将最大加数n_1不大于m的划分个数记作q(n,m)。可以建立q(n,m)的如下递归关系。

(3) q(n,n)=1+q(n,n-1);
正整数n的划分由n_i =n的划分和n_i ≤n-1的划分组成。

(4) q(n,m)=q(n,m-1)+q(n-m,m),n>m>1;　比如：q(6,2)
正整数n的最大加数n_i不大于m的划分由:
n_i ≤m-1 的划分(全部划分<M)和n_i =m的划分组成。
（不包含m和至少包含一个m两种划分）
q(n-m,m)：即去掉一个m后n-m的可能划分数
(含m的可能，当n>=m+m时)



整数划分问题

$$q(n,m)=\begin{cases}1 & n=1,m=1\\ q(n,n) & n<m\\ 1+q(n,n-1) & n=m\\ q(n,m-1)+q(n-m,m) & n>m>1\end{cases}$$

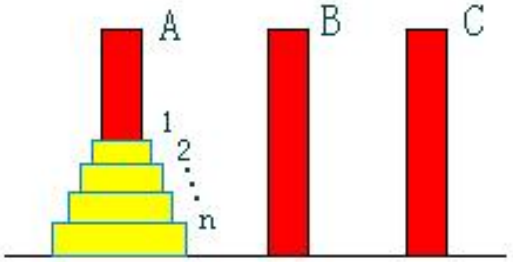
Hanoi塔问题

设a,b,c是3个塔座。开始时，在塔座a上有一叠共n个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为1,2,…,n,现要求将塔座a上的这一叠圆盘移到塔座b上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

规则1：每次只能移动1个圆盘；

规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

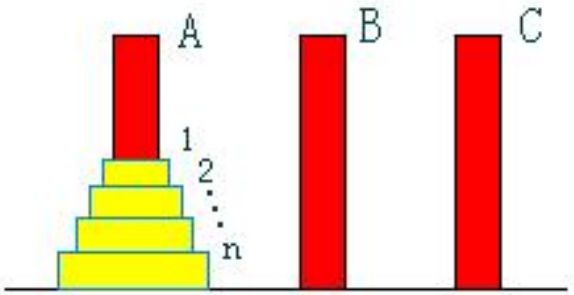
规则3：在满足移动规则1和2的前提下，可将圆盘移至a,b,c中任一塔座上。



Hanoi塔问题

```
void hanoi(int n, int a, int b, int c)
  {
    if (n > 0)
    {
      hanoi(n-1, a, c, b);
      move(a,b);
      hanoi(n-1, c, b, a);
    }
  }
```

2023年2月22日，厦门市滨北小学二年级学生郭弘奕成功打破一项世界纪录，以4.305秒的成绩，刷新“最快时间完成4层汉诺塔（单手）”WRCA世界纪录，成为该项目新世界纪录保持者。

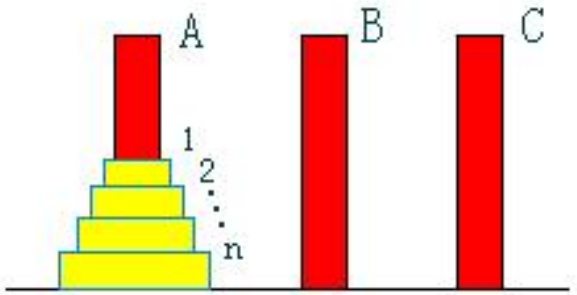


Hanoi塔问题

```c
#include <stdio.h>

void Hanoi(int n,char current,char transit,char target){
    if(n == 1){
        printf("%d %c %c\n",n,current,target);
    } else{
        Hanoi(n - 1,current,target,transit);
        printf("%d %c %c\n",n,current,target);
        Hanoi(n - 1,transit,current,target);
    }
}
int main() {
    int n;
    scanf("%d",&n);
    Hanoi(n,'A','C','B');
    return 0;
}
```

命名与可读性





分治法

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。



分治法

```
divide-and-conquer(P)  {
   if ( | P | <= n0) adhoc(P);                    //解决小规模的问题
   divide P into smaller subinstances P1,P2,...,Pk;    //分解问题
   for (i=1,i<=k,i++)                             //参考全排列的子问题/归并排序
      yi=divide-and-conquer(Pi);                  //递归的解各子问题
   return merge(y1,...,yk);                       //将各子问题的解合并为原问题的解
}
```

WHY?

在用分治法设计算法时，**最好使子问题的规模大致相同**。即将一个问题分成大小相等的k个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种**平衡(balancing)子问题**的思想，**它几乎总是比子问题规模不等的做法要好**。（如迅雷多线程下载的资源分配问题）

分治法的复杂性分析

一个分治法将规模为n的问题分成k个规模为n / m的子问题去解。设分解阀值n_0=1，且adhoc解规模为1的问题耗费1个单位时间。再设将原问题分解为k个子问题以及用merge将k个子问题的解合并为原问题的解需用f(n)个单位时间。用T(n)表示该分治法解规模为|P|=n的问题所需的计算时间，则有：

$$T(n)=\begin{cases} O(1) & n=1 \\ kT(n/m)+f(n) & n>1 \end{cases}$$

注意m和k的关系

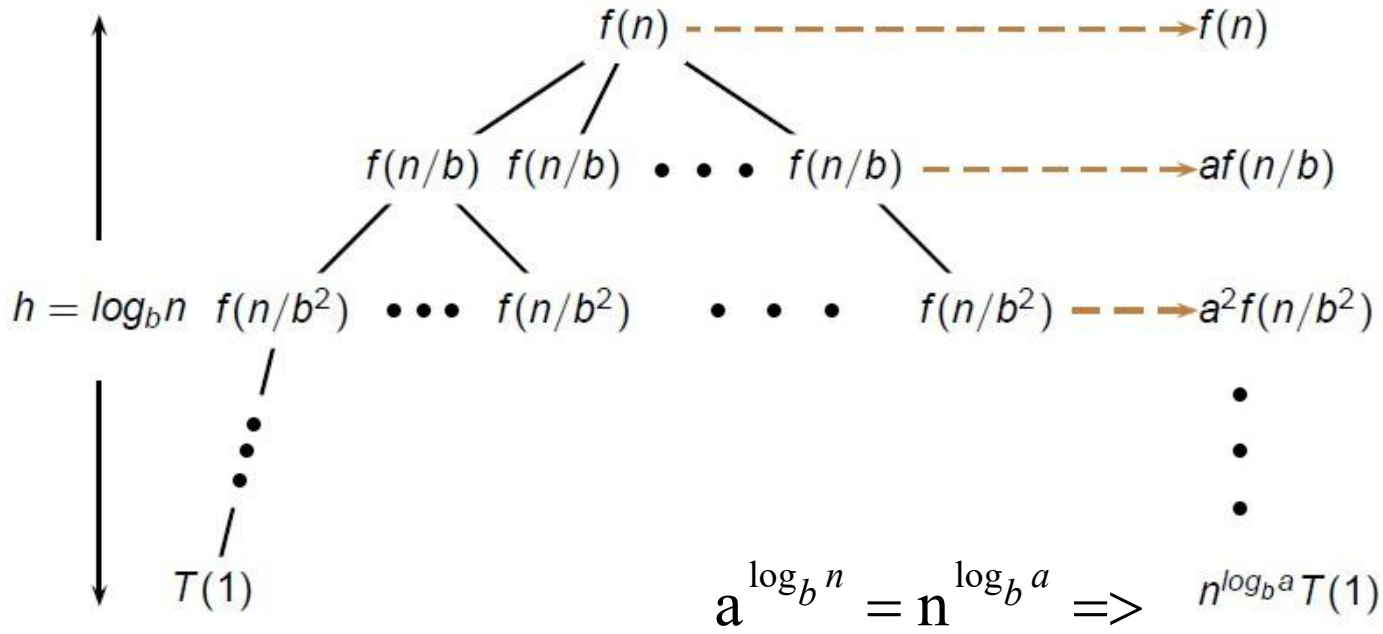
递归树多少层

通过迭代法求得方程的解：$T(n)=n^{\log_m k}+\sum_{j=0}^{\log_m n-1} k^j f(n/m^j)$

f为每层计算量
k为该层有多少个子问题

注意：递归方程及其解只给出n等于m的方幂时T(n)的值，但是如果认为T(n)足够平滑，那么由n等于m的方幂时T(n)的值可以估计T(n)的增长速度。通常假定T(n)是单调上升的，从而当$m^i \le n < m^{i+1}$时，$T(m^i) \le T(n) < T(m^{i+1})$。

分治法的复杂性分析



$$a^{\log_b n} = n^{\log_b a} \Rightarrow$$

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

二分查找技术



给定已按升序排好序的n个元素a[0:n-1]，现要在这n个元素中找出一特定元素x。

```
template<class Type>
int BinarySearch(Type a[], const Type& x, int l, int r){
    while (r >= l){
       int m = (l+r)/2;
       if (x == a[m]) return m;
       if (x < a[m]) r = m-1;
       else l = m+1;
    }
   return -1;
}
```

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了O(logn)次。循环体内运算需要O(1) 时间，因此整个算法在最坏情况下的计算时间复杂性为O(logn) 。

关于logn在复杂性中的物理意义

那么nlogn



二分查找技术

$$T(n) = T(n/2) + O(1)$$

$$T(n) = T(n/2) + O(1)$$
$$T(n) = T(n/4) + 2\,O(1)$$
$$T(n) = T(n/8) + 3\,O(1)$$
…

[共 logN 次]

$$T(n) = T(1) + \log N \cdot O(1)$$
$$T(n) = O(\log N)$$

二分查找技术



- K = 70

0	1	2	3	4	5	6	7	8	9	10	11	12
3	14	27	31	39	42	55	70	74	81	85	93	98
l						*m*						*r*
							l		*m*			*r*
							l,m	*r*				

大整数的乘法



设计一个有效的算法，可以进行两个n位大整数的乘法运算

◆小学的方法：$O(n^2)$ ✘效率太低

◆分治法:

$X =$

$Y =$

复杂度分析

$$T(n)=\begin{cases} O(1) & n=1 \\ 4T(n/2)+O(n) & n>1 \end{cases}$$

$T(n)=O(n^2)$ ✘没有改进

$$X = a\ 2^{n/2} + b \qquad Y = c\ 2^{n/2} + d$$

$$XY = ac\ 2^{n} + (ad+bc)\ 2^{n/2} + bd$$

大整数的乘法

$$XY = ac\ 2^n + (ad+bc)\ 2^{n/2} + bd$$

为了降低时间复杂度，必须减少乘法的次数。

1. $XY = ac\ 2^n + ((a-b)(d-c)+ac+bd)\ 2^{n/2} + bd$
2. $XY = ac\ 2^n + ((a+b)(d+c)-ac-bd)\ 2^{n/2} + bd$

复杂度分析

细节问题：两个XY的复杂度都是$O(n^{\log 3})$，但考虑到a+c,b+d可能得到m+1位的结果，使问题的规模变大。



大整数的乘法

- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。

- 最终的，这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。（可阅读参考材料）



Strassen矩阵乘法

A和B的乘积矩阵C中的元素C[i,j]定义为:

$$C[i][j]=\sum_{k=1}^{n}A[i][k]B[k][j]$$

若依此定义来计算A和B的乘积矩阵C，则每计算C的一个元素C[i][j]，需要做n次乘法和n-1次加法。因此，算出矩阵C的 个元素所需的计算时间为$O(n^3)$



Strassen矩阵乘法

使用与上例类似的技术，将矩阵A，B和C中每一矩阵都分块成4个大小相等的子矩阵。由此可将方程C=AB重写为：

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

由此可得：$C_{11} = A_{11}B_{11} + A_{12}B_{21}$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$$

Strassen矩阵乘法



为了降低时间复杂度，必须减少乘法的次数。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$M_1 = A_{11}(B_{12} - B_{22})$$
$$M_2 = (A_{11} + A_{12})B_{22}$$
$$M_3 = (A_{21} + A_{22})B_{11}$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$
$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$$

$T(n)=O(n^{\log 7})=O(n^{2.646})$



Strassen矩阵乘法

- 传统方法：$O(n^3)$
- 分治法: $O(n^{2.646})$
- 更快的方法??

➢Hopcroft和Kerr已经证明(1971)，计算2个 2 × 2 矩阵的乘积，7次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算2×2矩阵的7次乘法这样的方法了。或许应当研究 3 × 3 或 5 × 5 矩阵的更好算法。

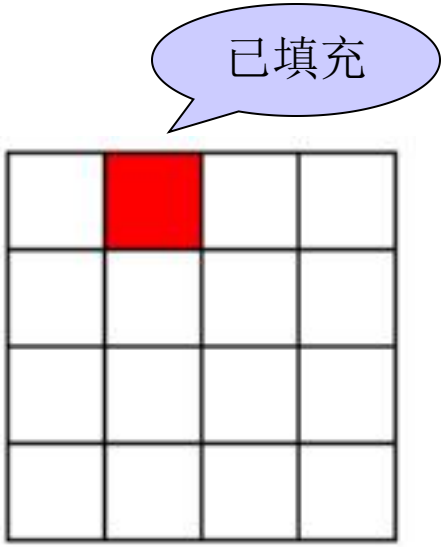
➢在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。目前?最好的计算时间上界是 **$O(n^{2.376})$**

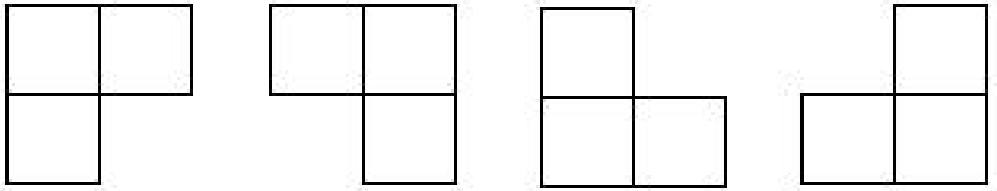
m	Bounds from Our Methods	Bounds from Prior Works	References
1	N/A	2.387190	[CW90]
2	2.374631	2.375477	[CW90]
4	2.371919	2.372927	[Sto10, Wil12, LG14]
8	2.371866	2.372865	[Wil12, LG14]
16	N/A	2.372864	[LG14]
32	N/A	2.372860	[AW21b]

➢是否能找到$O(n^2)$的算法?

棋盘覆盖

在一个$2^k \times 2^k$个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。

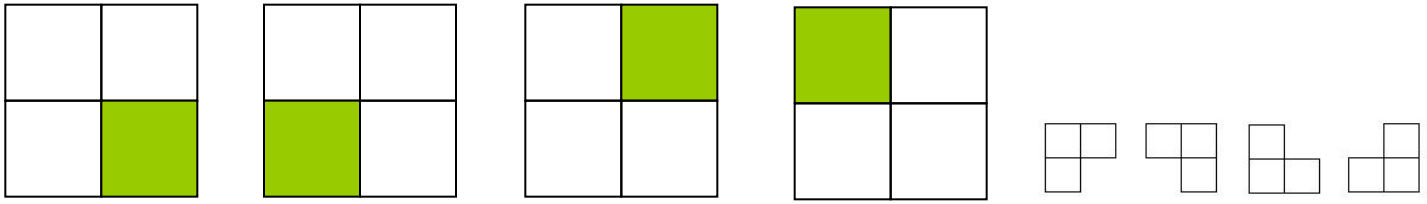


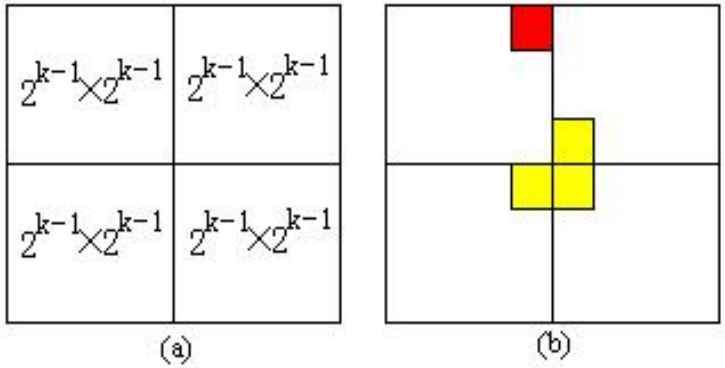


棋盘覆盖

关键字： 发现，最小尺寸

问题： 如何确保每个2*2中有个填充?





要求： 写出伪代码

棋盘覆盖

当k>0时，将$2^k \times 2^k$棋盘分割为4个$2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示。特殊方格必位于4个较小子棋盘之一中，其余3个子棋盘中无特殊方格。为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如 (b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘1×1。

复杂度分析

$$T(k)=\begin{cases} O(1) & k=0 \\ 4T(k-1)+O(1) & k>0 \end{cases}$$

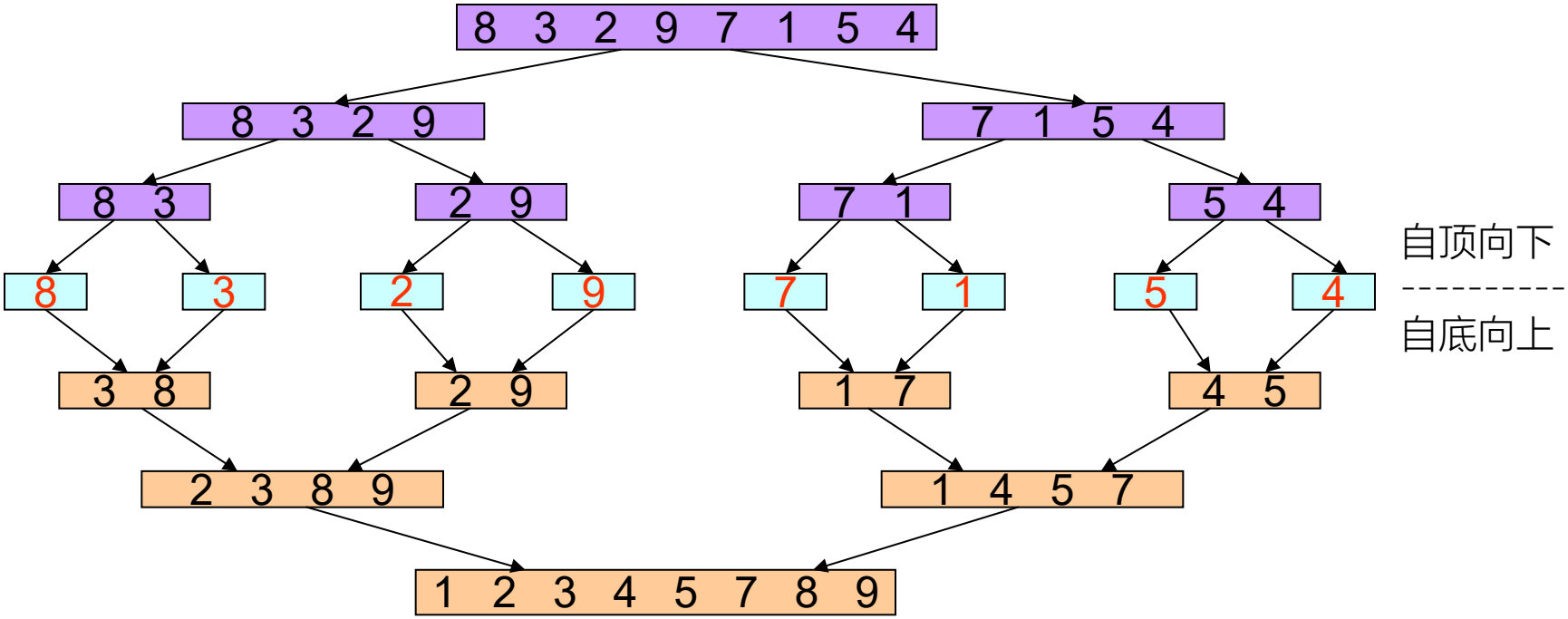
T(n)=O(4^k) 渐进意义下的最优算法

(a) (b)

合并排序

```
divide-and-conquer(P) {
  if ( | P | <= n0) adhoc(P);                    //解决小规模的问题
  divide P into smaller subinstances P1,P2,...,Pk ;   //分解问题
  for (i=1,i<=k,i++)                             //参考全排列的子问题
    yi=divide-and-conquer(Pi);                   //递归的解各子问题
  return merge(y1,...,yk);                       //将各子问题的解合并为原问题的解
}
```

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。



合并排序



```
static void merge(int array[], int p, int q, int r)
{
        int i,k;
        int begin1,end1,begin2,end2;
        int* temp = new int [r-p+1];      begin1= p;    end1 = q;
        begin2 = q+1;  end2 = r;
        k = 0;
        while((begin1 <= end1)&&( begin2 <= end2))          {
                if(array[begin1]<array[begin2])  {
                        temp[k] = array[begin1];  begin1++;
                }
                else {
                        temp[k] = array[begin2];  begin2++;
                }
                k++;
        }
        while(begin1<=end1){  //若第一个序列有剩余，直接拷贝出来粘到合并序列尾
                temp[k++] = array[begin1++];
        }
        while(begin2<=end2){  //若第二个序列有剩余，直接拷贝出来粘到合并序列尾
                temp[k++] = array[begin2++];
        }
        for (i = 0; i < (r - p +1); i++) //将排序好的序列拷贝回数组中
            array[p+i] = temp[i];
        delete[] (temp);
}
```

交叉插入排序

要求： 写出伪代码

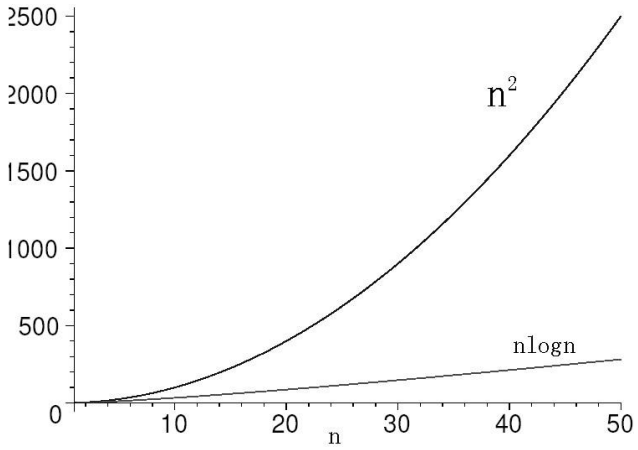
合并排序

复杂度分析

$$T(n)=\begin{cases} O(1) & n\le 1 \\ 2T(n/2)+O(n) & n>1 \end{cases}$$

T(n)=O(nlogn) 渐进意义下的最优算法

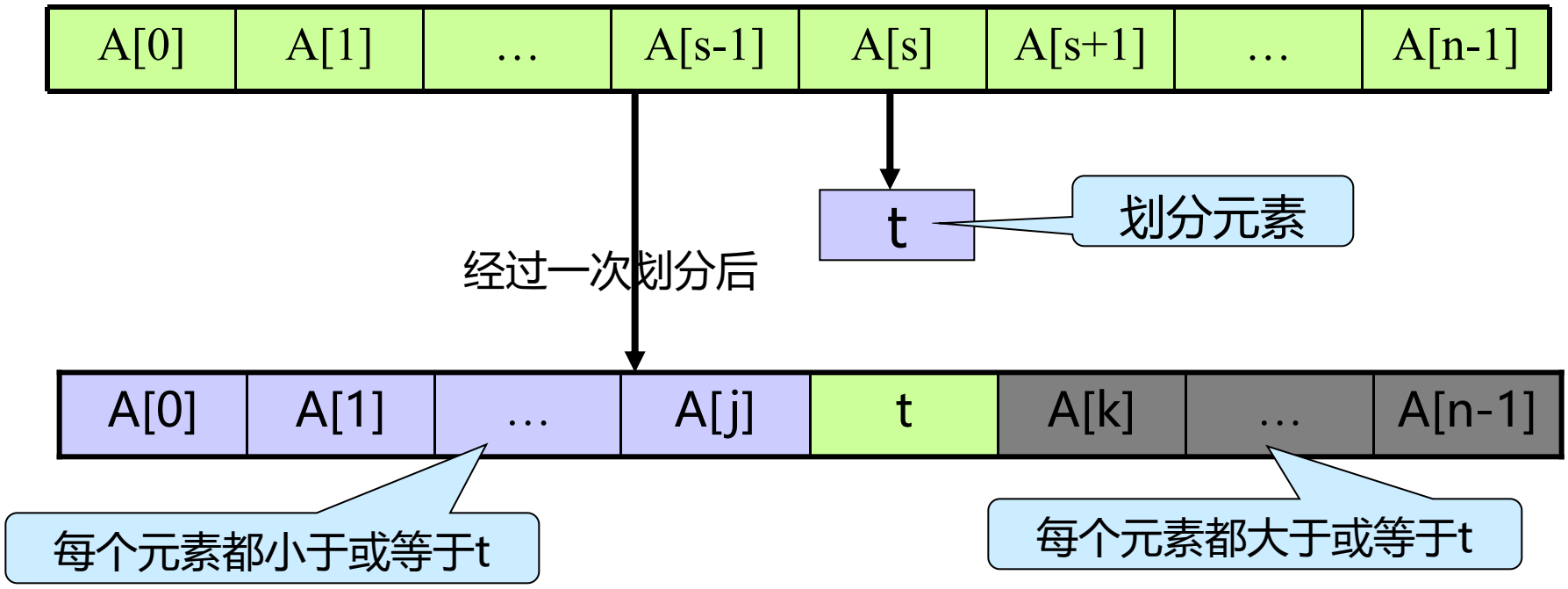
	Memory	Time
冒泡	300K	7483MS
合并	684K	171MS



- 最坏时间复杂度：**O(nlogn)**
- 平均时间复杂度：**O(nlogn)**
- 辅助空间：**O(n)**

快速排序

- 基本思想
  - 选取A的某个元素t=A[s]，然后将其他元素重新排列，使A[0..n-1]中所有在t以前出现的元素都小于或等于t，而在t之后出现的元素都大于或等于t。





快速排序

- 快速排序（Quicksort）是对冒泡排序的一种改进。
- 由C. A. R. Hoare在1962年提出。

- 它的基本思想是：通过一趟排序将要排序的数据分割成独立的两部分，其中一部分的所有数据都比另外一部分的所有数据都要小；
- 然后再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，以此达到整个数据变成有序序列。

快速排序的分段



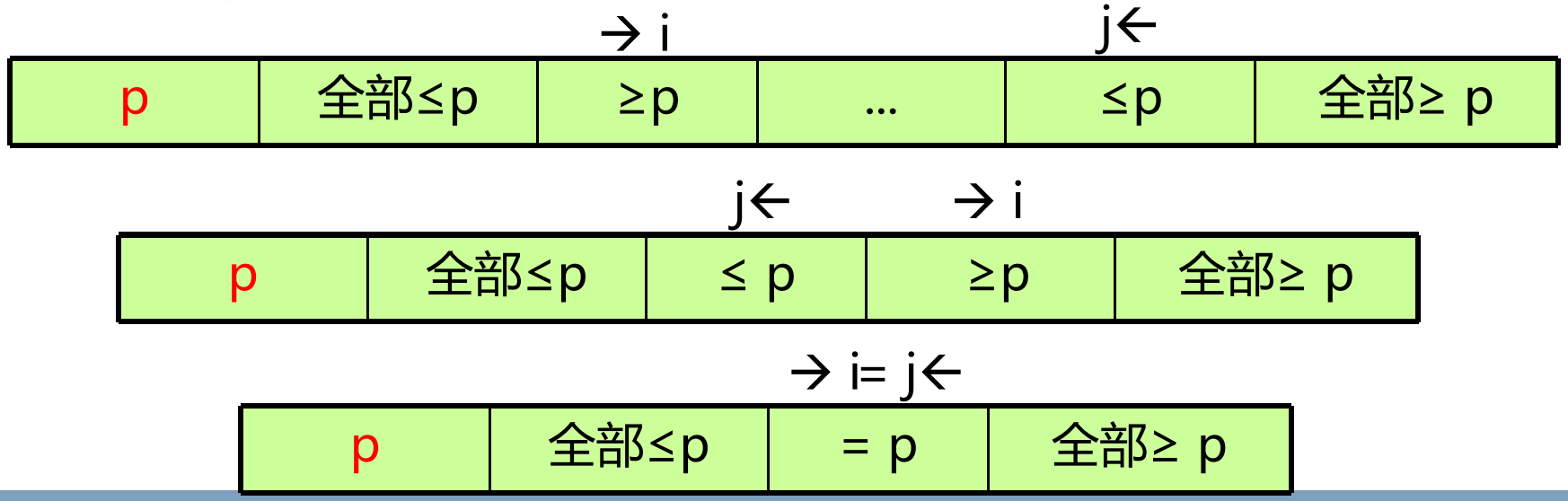
•划分元素（中轴）的选取：

•可以随机选取，最简单的选择策略是数组第一个元素(如何得到一个较好效率的划分元素？）**s ← Partition(A[l...r]);**

要求： 写出伪代码

•划分的实现思想

•一种方法（P不动）：同时从左、右开始扫描，左边找到一个比划分元素Pivot大的，右边找到一个比划分元素小的，然后交换两个元素的位置。



快速排序的分段

P（/S）会动：

算法 Quicksort(A[l...r])

//用Quicksort对子数组排序

//输入：数组A[0..n-1]中的子数组

//输出：非降序的字数组A[l...r]

```
if (l<r){
    s ← Partition(A[l...r]);
    Quicksort(A[l...s-1]);
    Quicksort(A[s+1...r]);
}
```

要求： 写出伪代码

{**6**, 7, 5, 2, $\bar{5}$, 8} 基准6

{**6**, 7, 5, 2, $\bar{5}$, 8} 不换j--;
 i j

{$\bar{5}$, 7, 5, 2, **6**, 8} 换i++;
 i j

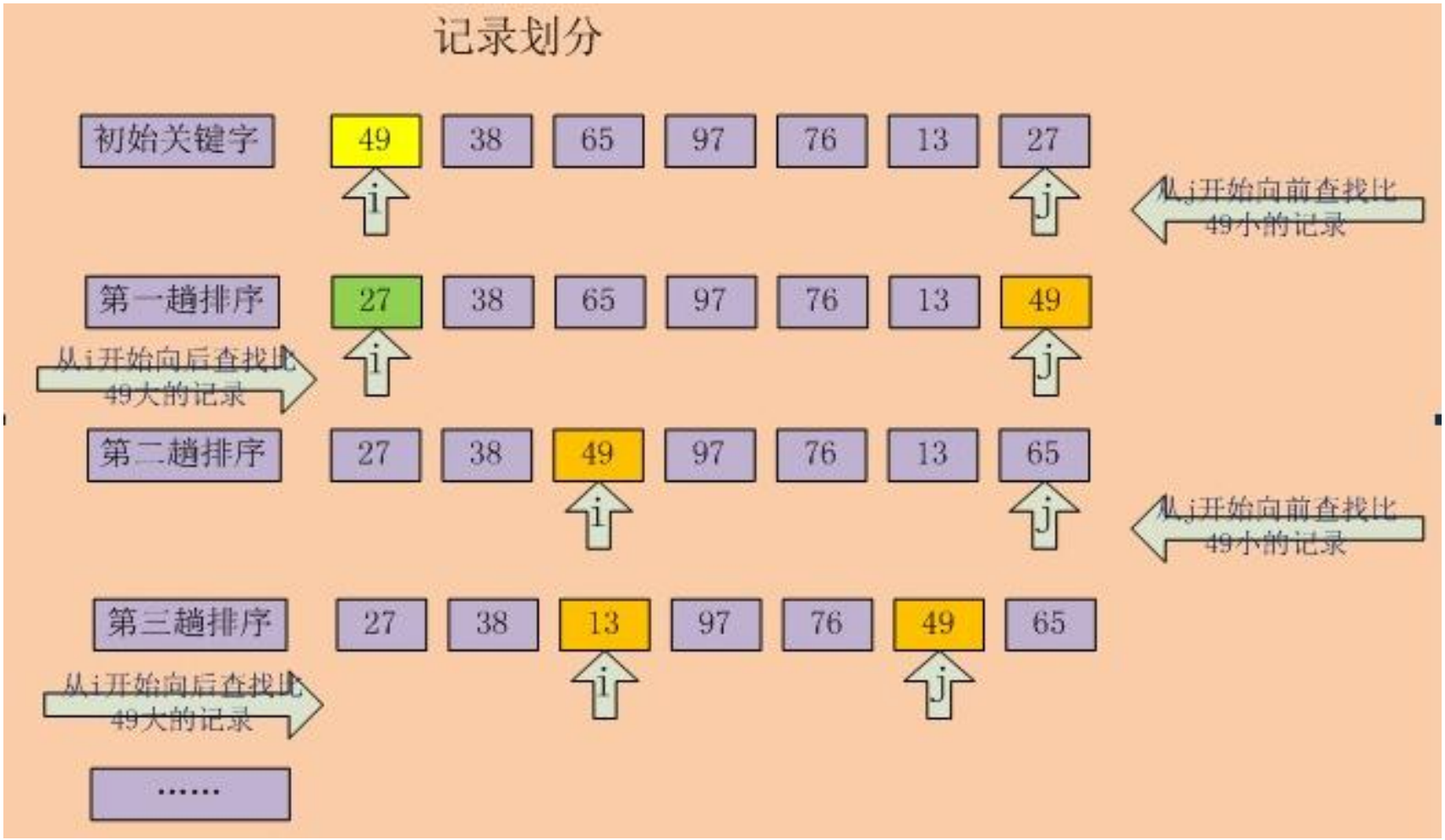
{$\bar{5}$, **6**, 5, 2, 7, 8} 换j--;
 i j

{$\bar{5}$, 2, 5, **6**, 7, 8}
 i j

{$\bar{5}$, 2, 5} **6** {7, 8} 完成

S：相对中位

快速排序


记录划分
初始关键字
49 38 65 97 76 13 27
i
j
从j开始向前查找比49小的记录
第一趟排序
27 38 65 97 76 13 49
从i开始向后查找比49大的记录
第二趟排序
27 38 49 97 76 13 65
从j开始向前查找比49小的记录
第三趟排序
27 38 13 97 76 49 65
从i开始向后查找比49大的记录
.....

快速排序

$$T_{best}(n)=\begin{cases}0 & n=1\\ 2T_{best}(n/2)+n & n>1\end{cases}$$

$T_{best}(n)=\Theta(n\log n)$

均衡：如果一半一半分（n/2）是最好的

$T_{worst}(n)=(n+1)+n+(n-1)+\ldots+3=\Theta(n^2)$

$$T_{avg}(n)=\begin{cases}0 & n=0,1\\ (\Sigma[(n+1)+T_{avg}(s)+T_{avg}(n-1-s)])/n & n>1\end{cases}$$

$T_{avg}(n) \approx 2n\ln n \approx 1.38n\log n$

线性时间选择

给定线性序集中n个元素和一个整数k，1≤k≤n，要求找出这n个元素中第k小的元素。

类似快速排序，只对其中一个部分进行递归处理。

{5, 2, 5} 6 {7, 8} 完成

找第2小
只需左半边

```
template<class Type>Type
RandomizedSelect(Type a[],int p,int r,int k)
{
    if (p==r) return a[p];
    int i=RandomizedPartition(a,p,r),
    //i的前面<a[i]；i的后面>a[i];
    j=i-p+1;
    if (k<=j) return RandomizedSelect(a,p,i,k);
    else return RandomizedSelect(a,i+1,r,k-j);
}
```

在最坏情况下，算法**randomizedSelect**需要$O(n^2)$计算时间
但可以证明，算法**randomizedSelect**可以在O(n)平均时间内找出n个输入元素中的第k小元素。



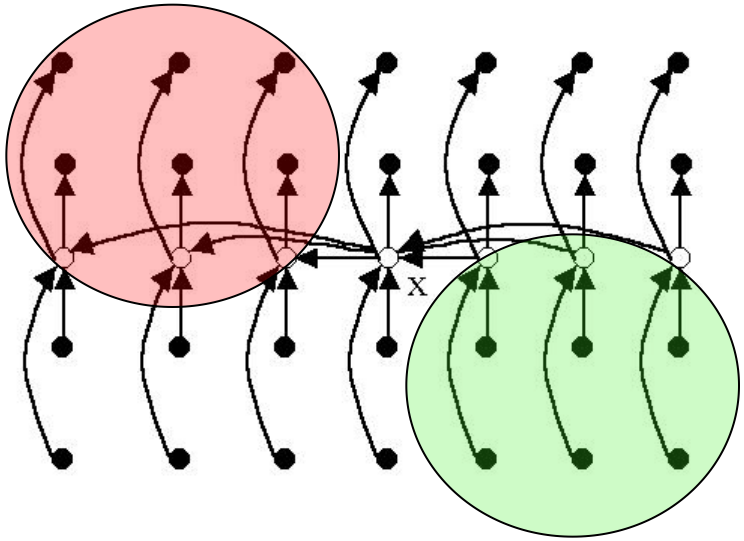
线性时间选择

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的ε倍(0<ε<1是某个正常数)，那么就可以**在最坏情况下**用O(n)时间完成选择任务。

例如，若ε=9/10，算法递归调用所产生的子数组的长度至少缩短1/10。所以，在最坏情况下，算法所需的计算时间T(n)满足递归式$T(n) \leq T(9n/10)+O(n)$ 。由此可得$T(n)=O(n)$。

线性时间选择：寻找划分基准

- 将n个输入元素划分成⌈n/5⌉个组，每组5个元素，只可能有一个组不是5个元素。用任意一种排序算法，将每组中的元素排好序，并取出每组的中位数，共⌈n/5⌉个。
- 递归调用**select**来找出这⌈n/5⌉个元素的中位数。如果⌈n/5⌉是偶数，就找它的2个中位数中较大的一个。以这个元素作为划分基准。



设所有元素互不相同。在这种情况下，找出的基准x至少比3(n-5)/10个元素大，因为在每一组中有2个元素小于本组的中位数，而n/5个中位数中又有(n-5)/10个小于基准x。同理，基准x也至少比3(n-5)/10个元素小。
而当n≥75时，3(n-5)/10≥n/4所以按此基准划分所得的2个子数组的长度都至少缩短1/4。

线性时间选择



- 算法见P27

复杂度分析

$$T(n) \le \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \ge 75 \end{cases}$$

T(n)=O(n)

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了T(n)的递归式中2个自变量之和n/5+3n/4=19n/20=εn，0<ε<1。这是使T(n)=O(n)的关键之处。当然，除了5和75之外，还有其他选择。

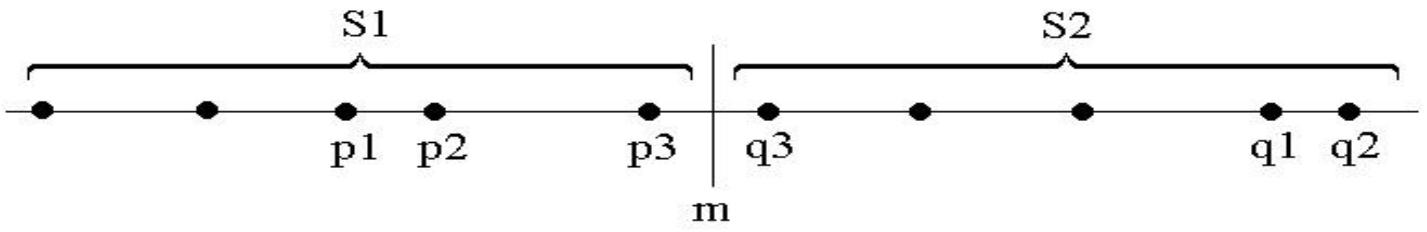
找中位数x：T(n/5)：按组找

下次递归：　T(3/4n)：至少减少1/4

最接近点对问题

为了使问题易于理解和分析，先来考虑一维的情形。此时，S中的n个点退化为x轴上的n个实数 $x_1,x_2,\ldots,x_n$。最接近点对即为这n个实数中相差最小的2个实数。

- 假设我们用x轴上某个点m将S划分为2个子集S1和S2，基于平衡子问题的思想，用S中各点坐标的中位数来作分割点。
- 递归地在S1和S2上找出其最接近点对{p1,p2}和{q1,q2}，并设**d=min{|p1-p2|,|q1-q2|}**，S中的最接近点对或者是{p1,p2}，或者是{q1,q2}，或者是某个{p3,q3}，其中p3∈S1且q3∈S2。
- 能否在线性时间内找到p3,q3?





最接近点对问题

- 如果S的最接近点对是{p3,q3}，即|p3-q3|<**d**，则p3和q3两者与m的距离不超过d，即p3∈(m-d,m]，q3∈(m,m+d]。

- 由于在S1中，每个长度为d的半闭区间至多包含一个点（否则必有两点且距离小于d），并且m是S1和S2的分割点，因此(m-d,m]中至多包含S中的一个点。由图可以看出，如果(m-d,m]中有S中的点，则此点就是S1中最大点。

- 因此，我们用线性时间就能找到区间(m-d,m]和(m,m+d]中所有点，即p3和q3。从而我们用线性时间就可以将S1的解和S2的解合并成为S的解。

最接近点对问题

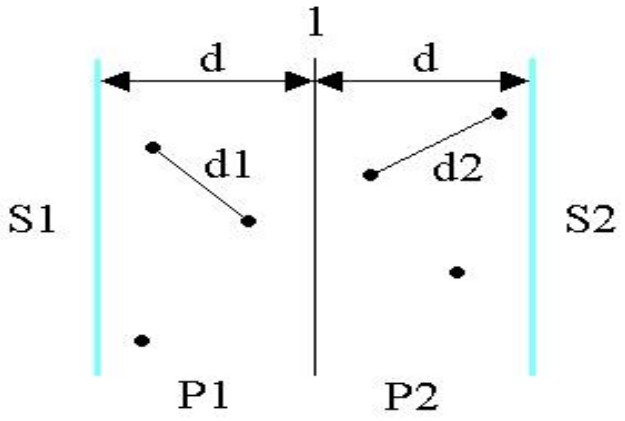
- 下面来考虑二维的情形。

➢选取一垂直线**l:x=m**来作为分割直线。其中m为S中各点x坐标的中位数。由此将S分割为S1和S2。

➢递归地在S1和S2上找出其最小距离d1和d2，并设**d=min{d1,d2}**

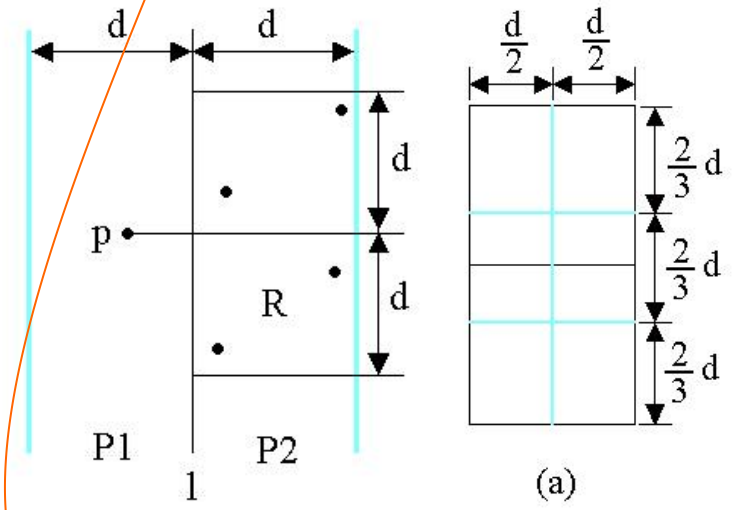
➢S中的最接近点对或者是d，或者是某个{p,q}，其中p∈P1且q∈P2。

➢**能否在线性时间内找到p,q?**



最接近点对问题

- 考虑P1中任意一点**p**，它若与P2中的点**q**构成最接近点对的候选者，则必有distance(p，q)＜d。**满足这个条件的P2中的点q一定落在一个d×2d的矩形R中**
- 由d的意义可知，P2中任何2个S中的点的距离都不小于d。由此可以推出**矩形R中最多只有6个S中的点**。
- 因此，在分治法的合并步骤中**最多只需要检查6×n/2=3n个候选者**



证明:将矩形R的长为2d的边3等分，将它的长为d的边2等分，由此导出6个(d/2)×(2d/3)的矩形。若矩形R中有多于6个S中的点，则由**鸽巢原理**易知至少有一个(d/2)×(2d/3)的小矩形中有2个以上S中的点。设u，v是位于同一小矩形中的2个点，则

$$(x(u)-x(v))^2+(y(u)-y(v))^2 \le (d/2)^2+(2d/3)^2 = \frac{25}{36}d^2$$

distance(u,v)<d。这与d的意义相矛盾。

最接近点对问题

鸽巢原理：
如果有n+1个鸽子飞进了n个鸽巢中，
那么必定有鸽巢中至少飞进了2只鸽子。

鸽巢原理加强版：
当n只鸽子飞进m个巢时，必定至少有一个巢中飞进了：

$$r = \lfloor (n-1)/m \rfloor + 1$$



最接近点对问题



- **为了确切地知道要检查哪6个点，可以将P和P2中所有S2的点投影到垂直线l上**。由于能与p点一起构成最接近点对候选者的S2中点一定在矩形R中，所以它们在直线l上的投影点距p在l上投影点的距离小于d。由上面的分析可知，这种投影点最多只有6个。
- 因此，**若将P1和P2中所有S中点按其y坐标排好序**，则对P1中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选者。对P1中每一点最多只要检查P2中排好序的相继6个点。

复杂度分析

$$T(n)=\begin{cases} O(1) & n<4 \\ 2T(n/2)+O(n) & n\geq 4 \end{cases}$$

T(n)=O(nlogn)

循环赛日程表

设计一个满足以下要求的比赛日程表：
(1)每个选手必须与其他n-1个选手各赛一次；
(2)每个选手一天只能赛一次；
(3)循环赛一共进行n-1天。

- 算法见P27
- n：块越来越少
- m：块越来越大

按分治策略，将所有的选手分为两半，n个选手的比赛日程表就可以通过为n/2个选手设计的比赛日程表来决定。递归地用对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1



循环赛日程表

```
void GameTable(int k,int array[][N])//数组下标从1开始
{
    int i,j,s,t;
    int n=1;
    for(i=1;i<=k;i++)
        n*=2;                          //求总人数
    for(i=1;i<=n;i++)
        array[1][i]=i;                 //第一行排1-8
    int m=1;                           //用来控制每一次填表时i行j列的起始填充位置
    for(s=1;s<=k;s++)                  //s指对称赋值的总循环次数，即分成几大步进行制作日程表
    {
        n=n/2;
        for(t=1;t<=n;t++)              //t指明内部对称赋值的循环次数
            for(i=m+1;i<=2*m;i++)
                for(j=m+1;j<=2*m;j++)
                {
                    array[i][j+(t-1)*m*2]=array[i-m][j+(t-1)*m*2-m];     //右上角等于左上角的值
                    array[i][j+(t-1)*m*2-m]=array[i-m][j+(t-1)*m*2];     //左下角等于右上角的值
                }
        m*=2;
    }
```

求出n（块）

第一列

n：块越来越少
m：块越来越大
含t的for循环表示对每一块操作



作业要求

题目：
acm.nbu.edu.cn
循环、字符串、数组
各完成10题
告诉账号或加入group